Temper: Falsify-or-Certify Assurance for Every Transformation from Trained Network to Edge Binary

Vaishnavi
Department and institution (fill in)
Email (fill in)

***Abstract*—Between a trained neural network and the program that runs on a device lie many transformations: compiler rewrites, hand edits, quantisation and pruning, and code generation. Each can silently change predictions, yet testing can only fail to find such bugs and verifiers can only fail to prove their absence. We present *Temper*, which gives every transformation a three-valued verdict: REFUTED with a replayable, localised counterexample; CERTIFIED with a proof that the outputs are bit-identical (or within ε) for every input of the deployment domain under float32 semantics; or UNDECIDED with the share of the domain already proven. Temper couples a boundary-targeting falsifier with a differential certifier that is sound for float32, tracks *exactness* to prove bit-identity, and statically locates *divergence sources* into which the falsifier solves for inputs. On 2,352 transformations of 96 networks, Temper decides 99.7% (84.6% and 84.2% for the falsifier and the certifier alone) with half the falsifier's evaluations and no unsound certificate; 54 of the faults it refutes are exposed by no input of a 4,000-input random oracle. On 80 faults modelled on real deep-learning compiler bug classes in 8 trained classifiers, it refutes all 71 observable ones (62 for test-set checking) and proves the other 9 harmless. Beyond the verdict, Temper brackets compression risk, repairs compressions by a certified per-layer precision search (up to 29% smaller than the best certified uniform format), localises a breaking edit to the exact weight, and re-certifies new model versions from previous proofs.**

***Index Terms*—ML compilers, translation validation, differential verification, model compression, falsification, certification, edge AI**

I. Introduction

A network trained in Python reaches a microcontroller or phone through a *chain* of transformations. An ML compiler fuses operators, folds constants and removes dead code [1], [2]; engineers hand-optimise or convert models; the weights are quantised or pruned; code is generated; and a C++ compiler applies its own optimisations, sometimes with `-ffast-math`. Any link can change a prediction, and production compilers do have such bugs [3].

Two families of tools address individual links. *Testers* (differential testing, fuzzing [3], [4]) are cheap but can only say "no bug found". *Verifiers* prove rewrite rules [5], single compilations [6], [7], [8] or properties of networks [9], [10], [11] but often reason in real arithmetic, need a formal model of every operator, and simply time out or say "unknown" when a bug exists. Neither gives a deployment engineer a single, actionable answer for the whole chain.

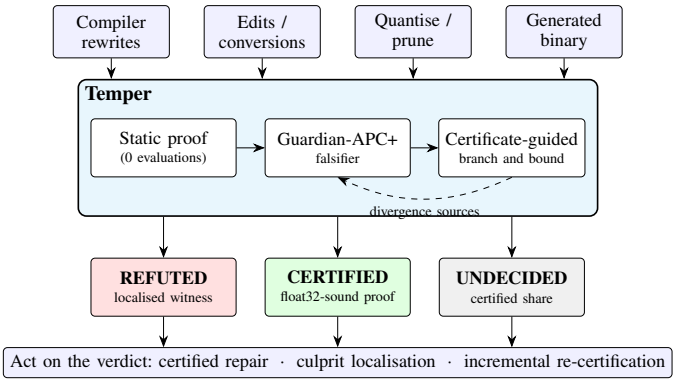


Fig. 1. Overview. Every transformation of a model receives one of three verdicts; the certifier tells the falsifier where a bug can live, and each verdict feeds a follow-up action.

Temper answers one question per transformation T of a model f into $f' = T(f)$, over a deployment domain D: *can the shipped program behave differently from the model I validated?* The answer is one of

- **REFUTED**: an input $x \in D$ with $f(x) \neq f'(x)$ beyond tolerance, minimised and localised to the first differing IR value;
- **CERTIFIED**: a proof that $f'(x) = f(x)$ *bit for bit* (or $\|f'(x) - f(x)\|_\infty \leq \varepsilon$) for all $x \in D$, under float32 semantics;
- **UNDECIDED**: neither, with the certified share of D.

The ModelForge compiler is one component: it produces candidate transformations and code, and Temper decides which to keep.

Fig. 1 summarises the approach.

Contributions.

1) A **falsify-or-certify** decision procedure for neural-network transformations that runs a static proof, a boundary-targeting falsifier (Guardian-APC+), and a certificate-guided branch and bound in sequence, each step informing the next (§V).
2) A **float32-sound, exactness-tracking differential certifier**: symbolic bounds padded by the dot-product rounding bound, a canonical form, lockstep propagation of $\Delta = f' - f$ with symbolic Δ bounds, and a per-

value *exactness bit* that proves bit-identity for structural rewrites (§IV).

3) **Divergence sources**: the certifier reports the units and pre-activation intervals where a candidate's operator can disagree with the original's; the falsifier Newton-solves inputs into them. This refutes faults that random testing at 4,000 inputs and Guardian-APC+ both miss, typically in a handful of evaluations (§IV-D).
4) **One verdict for every link**: compression risk as a *severity bracket* (found $\leq$ true worst $\leq$ proven), certified domain specialisation, and a *binary audit* that compiles the generated C++ under several flag sets and replays it against the certified model; the generated program carries its certified domain and fingerprint (§VI).
5) **Acting on the verdict** (§VII): a *certified mixed-precision repair* that returns the smallest per-layer int4/int8/fp16/fp32 assignment whose risk is proven low; *certified culprit localisation*, delta debugging [12] whose oracle is a Temper decision, so the reported fix is itself certified; and *incremental re-certification* from reusable, file-backed certificates that carry the branch-and-bound partition and the last counterexample, sound by construction because every reused leaf is re-verified.
6) An open, terminal-first implementation (C++17, no dependencies): a one-command deployment dossier with CI exit codes, an interactive studio, and a benchmark that regenerates every number here (§VIII).

II. Setting and Definitions

Models are lowered to an SSA IR over Gemm, MatMul, Add, ReLU, Sigmoid, Softmax and fused Gemm+ReLU. The IR interpreter and the generated C++ use the same float32 evaluation order. A *transformation* maps an IR graph f to f' with the same input shape. The *domain* D is a box: by default $[-10, 10]^n$, or the per-feature envelope of the deployment data widened by a margin.

Definition 1 (Exposure)**.** *x exposes f' if $\|f'(x) - f(x)\|_\infty > \tau$ or the predicted classes differ ($\tau = 10^{-5}$).*

Definition 2 (Certificates)**.** *f' is* bit-exact *on D if $f'(x) = f(x)$ in float32 for all $x \in D$, and* ε-bounded *if $\|f'(x) - f(x)\|_\infty \leq \varepsilon \leq \tau$.*

A bit-exact certificate excludes every exposure. An ε-bounded one excludes value exposures; a prediction can then change only where the top two scores are within 2ε.

III. The Falsifier: Guardian-APC+

Guardian-APC+ generates inputs from the model itself: interval bound propagation (IBP) selects ReLU units whose kink is reachable; Newton steps on the exact forward-mode input gradient place inputs on each kink at any depth and at fixed offsets on both sides; inputs are ordered by activation-pattern coverage, a submodular objective with a $(1 - 1/e)$ greedy guarantee; a structural diff of the IR focuses inputs on the activations a rewrite touches; and sub-tolerance output differences drive a near-miss hill climb. A rejected rewrite is minimised and localised to the first IR value that differs. In an equal-budget study (96 models, 2,160 faults, 128 inputs) it detects 99.2% of the faults a 4,000-input oracle observes, against 75.0% for uniform random testing (exact McNemar $p < 10^{-4}$), and 88.2% within 8 inputs. On its own it can only refute; Temper adds the other half.

IV. The Certifier

A. Float32-sound symbolic bounds

For a box B every value v receives concrete bounds and linear bounds $L_v(x) \leq v \leq U_v(x)$ in the box inputs (DeepPoly-style forward relaxations [10], [13], intersected with IBP [14]). For an affine output $y = b + \sum_{k=1}^{m} w_k a_k$ we pad both bounds by the classic rounding bound for a recursively summed dot product [15],

$$|\mathrm{fl}(y) - y| \leq \gamma_{m+1}\Big(|b| + \sum_k |w_k| \max |a_k|\Big), \quad \gamma_j = \tfrac{ju}{1-ju}, \tag{1}$$

with $u = 2^{-24}$, and exp-based operators by a few ulp. The bounds therefore enclose the values the float32 program computes, so "stable inactive" ($U \leq 0$) means the ReLU output is exactly 0 in the executed program, not only in $\mathbb{R}$.

B. Canonical form and alignment

Both graphs are *canonicalised*: fused Gemm+ReLU is split, constant Add/ReLU/Sigmoid/Softmax are folded with the interpreter's own float formulas, and instructions that cannot reach the output are removed. Each step preserves float32 values bit for bit. The canonical graphs are then aligned instruction by instruction: dense with dense (shapes equal; biases may be missing on one side), Add with Add, and elementwise with elementwise; operands must correspond (aligned values, or constants of equal size). A 64-bit FNV fingerprint of the canonical form identifies the certified model.

C. Lockstep differential propagation

For aligned values we bound $\Delta_v = v' - v$ per element and record an *exactness bit* e_v. *Dense:* $\Delta y_j = \Delta b_j + \sum_k \big(w'_{kj}\Delta x_k + (w'_{kj} - w_{kj})x_k\big)$, bounded with x's bounds and padded by both rounding bounds; e_{y_j} holds if $\Delta b_j = 0$ and every term is exactly zero, i.e. e_{x_k} and ($w'_{kj} = w_{kj}$ or $x_k \equiv 0$ on B). Because both programs then add identical terms in the same order (zero terms leave a float sum unchanged), $y'_j = y_j$ bitwise. *ReLU:* if both pre-activations are stably inactive, $\Delta = 0$ exactly; if both are stably active, $\Delta = \Delta z$; otherwise $\Delta \in [\min(0, \underline{\Delta z}), \max(0, \overline{\Delta z})]$ (ReLU is monotone and 1-Lipschitz). *Different elementwise operators* g (candidate) and f (original): $g(z') - f(z) = [f(z') - f(z)] + [g(z') - f(z')]$; the second term is bounded exactly over the candidate's interval and is identically zero (exact) when the interval misses the region where $g \neq f$. *Softmax:* rows of the Jacobian sum to zero and $\sum_j |J_{ij}| = 2p_i(1-p_i)$, so $|\Delta s_i| \leq g_i(\max \overline{\Delta z} - \min \underline{\Delta z})$ with g_i the largest $p(1-p)$ over the hull of both logit boxes. Every Δ is also intersected with the independent bound $[\underline{v'} - \overline{v}, \overline{v'} - \underline{v}]$. As in ReluDiff [11], [16], Δ additionally carries symbolic bounds in the box inputs; they

Algorithm 1 Temper decision for $f \to f'$ on domain D

$c \leftarrow$ CERTIFY(f, f', D) ▷ one box, no evaluation
if c proves bit-exact or $\varepsilon \le \tau$ **then return** CERTIFIED
end if
$w \leftarrow$ GUARDIANAPC+(f, f', 96 targeted + 32 near-miss)
if $w \in D$ exposes f' **then return** REFUTED(w)
end if
$Q \leftarrow \{D\}$ ▷ largest box first
while $Q \neq \emptyset$ and budgets remain **do**
  $B \leftarrow$ pop(Q); $c \leftarrow$ CERTIFY(f, f', B)
  if c proves B **then**
    mark B certified; **continue**
  end if
  for $x \in$ {centre of B} $\cup$ SOLVE(sources(c)) $\cup$ VERTEX$(\nabla\Delta)$ **do**
    if x exposes f' **then return** REFUTED(x)
    end if
  end for
  split B along $\arg\max_i \text{width}_i \cdot \sum_j |W^{(1)}_{ij}|$; push halves
end while
return CERTIFIED if all boxes certified, else UNDECIDED(certified share)

pass through dense layers and through ReLU pairs that are active in both models, which retains cancellations that intervals lose. Unaligned graphs fall back to the symbolic difference $L_{f'} - U_f \le \Delta \le U_{f'} - L_f$.

Theorem 1 (Soundness)**.** *If the analysis of B returns exactness for every output, then $f'(x) = f(x)$ in float32 for all $x \in B$; if it returns $|\Delta| \le \varepsilon$, then $\|f'(x) - f(x)\|_\infty \le \varepsilon$ for all $x \in B$.*

Proof sketch. By induction over the aligned instructions: each rule encloses the float32 difference given enclosures of the operands (bounds are padded for rounding), and each exactness rule implies that both programs perform the same float operations on equal operands or add exact zeros. □

D. Divergence sources

When an elementwise pair (f, g) differs only on a region R of pre-activations (a dead zone $(0, p)$, a clamp above p, a leaky slope below 0), every unit whose candidate interval meets R is a *divergence source* $(v, i, R \cap [\underline{z'_i}, \overline{z'_i}])$. Exactness propagation shows that *only* these units can make the outputs differ. Sources are what the falsifier should aim at, and the certifier hands them over: Newton iterations on the candidate's exact input gradient move the input until z'_i lies at the middle (then 0.9, 0.1) of the interval, inside the current box.

V. Falsify or Certify

Algorithm 1 orders the steps by cost. The static proof is a single bound computation and settles every structural rewrite; the falsifier settles almost every real bug within a few evaluations; the branch and bound spends its budget only on what is left. Inside it, each undecided box contributes three witnesses: its centre, Newton-solved inputs for its divergence sources, and the box vertex that maximises the linearised difference ($\nabla f'_k - \nabla f_k$ at the centre, for the output with the widest Δ bound). The idea of letting proof attempts guide test generation follows SYNERGY [17] and CEGAR [18]; here the abstraction is a differential one over two networks, and the guidance is a set of value ranges to solve into.

VI. Every Link of the Chain

Compiler passes and model edits. ModelForge keeps a pass only if it is not refuted; in strict mode only if it is certified. `temper a.mforge --compare b.mforge` decides whether an edited, converted or hand-optimised model is equivalent.

Compression risk as a bracket. For a compressed f' the right question is not equivalence (it never holds) but *how confident can the original be at an input where the two disagree?* Let $\mathcal{R}$ be the union of L_∞ balls of radius 0.5 (in standard deviations) around the test rows, a realistic region.

Proposition 1. *If $|p'_i(x) - p_i(x)| \le \varepsilon$ for all i and $x \in B$, then at every $x \in B$ where $\arg\max p' \neq \arg\max p$, the original's top-two margin is at most 2ε.*

Proof. With $c = \arg\max p$ and $p'_j \ge p'_c$: $p_c - p_j \le (p'_c + \varepsilon) - (p'_j - \varepsilon) \le 2\varepsilon$. □

Per ball, branch and bound proves either prediction agreement on a box (logit margins of both models positive, using symbolic margins and Δ) or a local bound 2ε; the maximum over $\mathcal{R}$ is the *proven* severity. Projected sign-gradient attacks on both models' margins and on their gap, followed by a hill climb, give the *found* severity. The true worst case lies between them; a compression is CERTIFIED low-risk if the proven severity is below 0.2 and REFUTED if the found one is at least 0.2, the risk scale of our earlier boundary-shift study.

Domain specialisation. ReLU units proven inactive on D (float32-sound) are removed with their weights; the masked model is re-certified bit-exact by the certifier.

The binary. The generated C++ is compiled with the system compiler under `-O0`, `-O2`, `-O3 -march=native -ffp-contract=fast` and `-O3 -ffast-math`, run on the test rows and the Guardian inputs, and compared with the interpreter score by score. The generated program embeds the certified domain and the model fingerprint and reports, per input, whether it lies inside the region where the certificates hold.

VII. After the Verdict: Repair, Localise, Re-certify

A verdict is most useful when it says what to do next.

Certified mixed-precision repair. Uniform formats are coarse: when int8 is refuted or undecided, the engineer falls back to fp16 for the whole model. Mixed-precision quantisation [19] chooses per-layer bit widths from a sensitivity proxy; Temper chooses them from a proof. Weight matrices are visited largest first; each is lowered to the fewest bits in $\{4, 8, 16\}$ for which the *whole* model's proven severity (§VI)

TABLE I
VERDICTS ON 2,352 TRANSFORMATIONS ($D = [-10, 10]^n$). R/C/U: REFUTED, CERTIFIED, UNDECIDED. UNSOUND: CERTIFIED CASES EXPOSED BY ANY INPUT.

Strategy	Observable refuted	Benign certified	Hidden R/C/U	All decided	Evals /case	Unsound
Guardian-APC+	96.7%	0%	0/0/6	84.6%	23.5	–
Certifier (B&B)	82.3%	100%	0/0/6	84.2%	101.8	0
Temper	**99.95%**	**100%**	0/0/6	**99.7%**	**11.3**	**0**

stays below the low-risk threshold, the others keeping their current widths. Every accepted step is a certificate, so the returned plan is CERTIFIED by construction; the full bracket is then recomputed for the report. `temper model --data test.csv --repair` writes the repaired model.

Certified culprit localisation. When an edit is REFUTED and both models have the same canonical structure (§IV; a hand-fused copy qualifies), Temper lists the changed parameters and runs ddmin [12] over them with the oracle "the original plus this subset of edits is REFUTED". The result is a 1-minimal failure-inducing subset. Temper then reverts exactly that subset in the candidate and decides again, so the suggested fix comes with a verdict of its own. Unlike spectrum-based fault localisation for networks [20], the culprit is a set of parameter edits and the evidence is a counterexample and, after the revert, a proof.

Incremental re-certification. Models change often: retraining, patches, re-export. A Temper certificate stores both canonical fingerprints, the domain, the final branch-and-bound partition (which always covers the domain, whatever the verdict) and, if refuted, the witness. For the next version, Temper first replays the stored witness: a regression is refuted in one evaluation. Otherwise branch and bound starts from the stored partition rather than the root, as IVAN does for single-network verification [21]. Each leaf is re-verified, so a stale certificate costs time but never soundness.

VIII. EVALUATION

All runs: one core, GCC 13.3, Linux, master seed 20260917; the whole benchmark takes about four minutes.

A. RQ1: How many transformations get a verdict?

Corpus. The 96 seeded classifiers of the Guardian study (1 to 4 hidden ReLU layers, widths 4 to 16) with 2,064 injected faults (dead zones of three fixed and one random width, upper clamps, leaky ReLU, ReLU→Sigmoid, 10^{-3} weight and 10^{-2} bias perturbations, a dropped bias, Softmax→Sigmoid), 96 negative controls (a rewrite of an output-unreachable node) and 192 real optimisation passes (fusion and dead-code removal). A fault is *observable* if a 4,000-input random oracle or any verified witness exposes it.

Table I: Temper refutes 2,057 of the 2,058 observable faults and certifies all 288 benign transformations bit-exact, all of them by the static proof without a single model evaluation. The falsifier alone cannot certify anything; the certifier alone refutes only what its box centres happen to hit. No certificate was ever contradicted. Six faults (10^{-3} weight and 10^{-2} bias perturbations) are exposed by nothing and remain UNDECIDED: their output change stays near τ, too small to refute and too close to prove.

TABLE II
SEVERITY BRACKETS (FOUND..PROVEN WORST-FLIP MARGIN). C: CERTIFIED LOW RISK; R: REFUTED (FOUND ≥ 0.2); U: UNDECIDED.

Model	fp16	int8	int4	prune30	prune50
BC (16)	.00..01 C	.02..17 C	.79..1 R	.43..1 R	.71..1 R
BC (32,16)	.00..03 C	.03..1 U	.81..1 R	.78..1 R	.99..1 R
Digits (16)	.00..02 C	.16..78 U	1..1 R	1..1 R	1..1 R
Digits (32,16)	.00..08 C	.23..1 R	1..1 R	1..1 R	1..1 R
Iris (16)	.00..00 C	.01..03 C	.53..99 R	.36..71 R	.85..1 R
Iris (32,16)	.00..00 C	.01..07 C	.40..1 R	.53..1 R	.91..1 R
Wine (16)	.00..00 C	.00..09 C	.26..1 R	.49..1 R	.64..1 R
Wine (32,16)	.00..01 C	.01..32 U	.49..1 R	.61..1 R	.93..1 R

B. RQ2: Does certification help falsification?

Guardian-APC+ misses 68 observable faults; Temper refutes 67 of them. For 54 of these, no input of the 4,000-input oracle exposes the fault: the only witness is a certificate-guided one. They are almost all random-width dead zones (as narrow as 10^{-4}), where the faulty region is a thin slab of input space: Guardian-APC+ catches 176 of 240, the unguided certifier 113, Temper all 240. The divergence source names the unit and the interval $(0, p)$; one Newton solve places the input inside it.

C. RQ3: Is a compressed model safe to ship?

Eight scikit-learn ReLU MLPs (hidden layers (16) and (32,16)) on Iris, Wine, Breast Cancer and Digits, each compressed five ways (float16; int8 and int4 symmetric per-tensor weights; 30% and 50% magnitude pruning): 40 cases, every test row as a ball centre (45 to 540 per model).

Table II: 12 compressions are certified low-risk (every float16 and four int8), 25 refuted, 3 undecided, and no found severity ever exceeds its proven bound. Seven refuted compressions kept their full test accuracy, and four changed no test prediction at all: Wine with int4 weights keeps 98.1% accuracy yet has a realistic input where the original's margin is 0.26 and the compressed model disagrees; for Breast Cancer (16) with int4 the margin is 0.79. The certified recommendation (smallest certified-low-risk compression) agrees with a search-only rule on four models and is more conservative on the other four, where int8 cannot yet be proven safe; on Breast Cancer (16) it now recommends int8, avoiding the int4 choice our earlier search-only tool made from a 200-evaluation search.

D. RQ4: Does the binary do what was certified?

Table III: with standard flags the binaries reproduce the certified model exactly, so certificates about the IR transfer to the shipped code. The audit also found a real defect in our own code generator: the generated CSV parser used `std::stof`, which rejects subnormal values such as 10^{-40} that a Guardian input contained, so one model's binary refused its input file; we replaced it with `strtof`. FMA contraction and fast-math

TABLE III
GENERATED C++ OF 8 REAL MODELS VS THE CERTIFIED MODEL (TEST ROWS AND GUARDIAN INPUTS, 173 TO 844 PER MODEL).

Flags	Bit-exact	Max $\lvert\Delta\rvert$	Prediction changes
`-O0`	8/8	0	0
`-O2`	8/8	0	0
`-O3 -march=native -ffp-contract=fast`	0/8	8.9×10^{-7}	0
`-O3 -ffast-math`	0/8	4.8×10^{-7}	0

TABLE IV
BUG CLASSES ON 8 REAL MODELS (NUMBER OF MODELS).

class	obs.	test set	random 1k	REFUTED	CERT.
channel swap (layout)	8	8	8	8	0
fusion drops bias	8	8	8	8	0
bias broadcast	8	8	8	8	0
loop off by one	8	8	8	8	0
LeakyReLU slope	8	8	8	8	0
ReLU lowered as ReLU6	7	4	5	7	1
ReLU as $x > 10^{-3}$	8	2	5	8	0
requant truncation	8	8	8	8	0
requant no saturation	8	8	8	8	0
requant tie rounding	0	0	0	0	8
control: optimiser	0	0	0	0	8

break bit-exactness on every model; the deviation stays below 10^{-6} and no prediction changes on these inputs, which the dossier reports as a warning rather than a blocker.

E. RQ5: Cost and usability

The decision study takes 46 s for 2,352 transformations (about 5.6 ms per case for Temper and for Guardian-APC+ alone: static proofs make benign cases nearly free). A full dossier for the Wine model (passes, specialisation, five compressions, four binaries) takes 3.7 s. `temper model.mforge --data test.csv` prints the dossier in the terminal, writes `dossier.json` and returns exit code 0 (ship) or 3 (blocked), so it slots into CI; `--compare` returns 0, 3 or 4 for certified, refuted and undecided.

F. RQ6: Does Temper catch realistic compiler bugs?

Studies of deep-learning compiler and converter bugs [22] report recurring root causes: wrong layouts, broken fusion, shape and broadcast errors, wrong loop bounds, activation lowering, and quantisation rounding and overflow. We model one fault per class (Table IV) and inject it into each of the 8 real classifiers, with the correct int8 model as reference for the quantisation classes. The domain is the test-data envelope widened by 0.5; a fault is *observable* if a 20,000-input random oracle or a Temper witness exposes it. Temper refutes all 71 observable faults (7.2 evaluations on average), against 62 for checking the test set and 66 for 1,000 random inputs. The two subtle classes, a ReLU6 clamp that only bites at large activations and an epsilon comparison that only bites in a thin slab, are the ones test sets miss. Temper also proves 9 faults *harmless on the domain*, bit for bit: rounding ties away from zero never changes a weight of these models, and in one model no ReLU input can reach 6. A tester can only say "not found". No certificate is contradicted by the oracle.

G. RQ7: Repair, localisation and incremental proofs

Repair. The certified per-layer search finds a CERTIFIED plan for all 8 models (6–12 severity proofs each). For Wine (32,16) it keeps 35.7% of the weight storage (fp16/int8/int8), against 50% for the best certified uniform format (fp16), at unchanged test accuracy; for Breast Cancer (32,16) it saves a little (49.5%); for the other six the best uniform format is already optimal. int4 is never certified for any layer. **Localisation.** In each model we hide one harmful edit, a sign flip of the largest first-layer weight, among 21–24 one-ulp edits. ddmin returns exactly the harmful weight in 8 of 8 models with 8–10 decisions (9 ms to 1.8 s). Reverting it is CERTIFIED ($\varepsilon = 10^{-3}$) in 5 models; in the three deepest models the remaining one-ulp edits are not refuted but not proven either. On the hand-optimised Iris example, Temper names the mistyped weight (`w2[8]`: $0.5 \to 0.505$) and certifies the revert. **Incremental re-certification.** Version chains (fp16 export, then two bias patches; $\varepsilon = 2 \cdot 10^{-3}$, 4,096 boxes): for certified chains, reuse halves the boxes (Iris (32,16): 761 to 384; Iris 16: 35 to 19); for refuted chains (4 models), the carried witness refutes each new version in one evaluation instead of 49–2,487 boxes; where fp16 is undecided at this budget, reuse neither helps nor hurts. Verdicts with and without reuse always agree.

IX. RELATED WORK

Translation validation and rewrite verification. Translation validation checks individual compiler runs [6], [7]; Alive2 does so for LLVM with SMT [8]; TensorRight proves tensor rewrite rules [5]. Temper validates each run of an ML compiler with abstract interpretation instead of SMT, proves bit-exactness under float32, and falls back to a falsifier when proof fails. **Differential verification.** ReluDiff and NeuroDiff bound the difference of two networks with symbolic intervals [11], [16], targeting compressed networks in real arithmetic. We add float32 rounding, exactness bits, operator-pair rules for differing activations, divergence sources, and a falsifier in the loop, and use the result for compiler passes, edits and compression alike. **Network verification.** DeepPoly, CROWN and symbolic intervals [10], [23], [13] and complete methods [9] certify properties of one network; quantised-network verification [24] addresses bit-precise semantics of one quantised model. We certify relations between two programs. **Testing.** NNSmith [3] and Csmith [25] generate programs; DiffChaser [4] searches for disagreements between a model and its compressed version; coverage criteria [26], [27] guide input generation, with known limits [28]. Combining proof attempts with directed testing goes back to DART and SYNERGY [29], [17]. **Repair, localisation and reuse.** HAWQ [19] picks per-layer precision from Hessian sensitivity, without guarantees; our search uses certified severity as the acceptance test. Delta debugging [12] isolates failure-inducing changes; DeepFault [20] ranks suspicious neurons; we isolate parameter edits with a certified oracle. IVAN [21] reuses a branch-and-bound tree when one network is modified; we reuse it for the *difference* of two networks and carry counterexamples forward.

Our bug classes follow the empirical study of deep-learning compiler bugs by Shen et al. [22].

X. Threats to Validity

Scale. Dense ReLU networks with up to 64 inputs and 32 hidden units per layer; convolutions, attention and real ONNX models are future work. Bounds loosen with depth, which is why int8 on the deeper Digits and Breast Cancer models stays undecided. **Fault realism.** Injected operators stand in for buggy kernels, and the bug classes of RQ6 are modelled on reported root causes, not reproduced from specific issues; divergence sources need an abstract semantics for each candidate operator, which a real compiler must provide. **Trusted base.** The canonicaliser, the interval rules and the interpreter are trusted; the interpreter and code generator share operator definitions, which the binary audit checks but does not prove. The binary audit is testing, not proof. **Thresholds.** The realistic radius (0.5 standard deviations, L_∞) and the risk threshold (0.2) are our choices; the found severity depends on search effort. **Specialisation.** The compact trained models have almost no provably dead units on their data envelope (one of 48 in Iris (32,16)); the benefit is expected for over-parameterised models, which we have not measured.

XI. Conclusion

Forging produces the program; tempering makes it trustworthy. Temper gives every transformation between a trained network and its binary a verdict that is either a counterexample or a proof, decides 99.7% of 2,352 transformations with no unsound certificate, finds faults that both random testing and a strong falsifier miss by solving into statically located divergence sources, brackets compression risk on real models, and checks the generated binary against the certified model. Everything runs from one terminal command.

Reproducibility

`cmake -S . -B build && cmake --build build --config Release`, then `modelforge_bench` (`--quick` for a short run). It prints every table in the terminal and writes `results.json`, `temper_results.json`, `compression_results.json`, `binary_results.json`, `lifecycle_results.json` and per-case CSV files (`--lifecycle-only` for RQ6–7).

References

[1] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, "TVM: An automated end-to-end optimizing compiler for deep learning," in *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[2] N. Rotem *et al.*, "Glow: Graph lowering compiler techniques for neural networks," *arXiv preprint arXiv:1805.00907*, 2018.

[3] J. Liu, J. Lin, F. Ruffy, C. Tan, J. Li, A. Panda, and L. Zhang, "NNSmith: Generating diverse and valid test cases for deep learning compilers," in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.

[4] X. Xie, L. Ma, H. Wang, Y. Li, Y. Liu, and X. Li, "DiffChaser: Detecting disagreements for deep neural networks," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence (IJCAI)*, 2019.

[5] J. Arora, S. Lu, D. Jain, T. Xu, F. Houshmand, P. M. Phothilimthana, M. Lesani, P. Narayanan, K. S. Murthy, R. Bodik, A. Sabne, and C. Mendis, "TensorRight: Automated verification of tensor graph rewrites," *Proceedings of the ACM on Programming Languages*, vol. 9, no. POPL, 2025.

[6] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 1998.

[7] G. C. Necula, "Translation validation for an optimizing compiler," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2000.

[8] N. P. Lopes, J. Lee, C.-K. Hur, Z. Liu, and J. Regehr, "Alive2: Bounded translation validation for LLVM," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021.

[9] G. Katz, C. Barrett, D. L. Dill, K. Julian, and M. J. Kochenderfer, "Reluplex: An efficient SMT solver for verifying deep neural networks," in *Computer Aided Verification (CAV)*, 2017.

[10] G. Singh, T. Gehr, M. Püschel, and M. Vechev, "An abstract domain for certifying neural networks," *Proceedings of the ACM on Programming Languages*, vol. 3, no. POPL, pp. 41:1–41:30, 2019.

[11] B. Paulsen, J. Wang, and C. Wang, "ReluDiff: Differential verification of deep neural networks," in *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*, 2020, pp. 714–726.

[12] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Transactions on Software Engineering*, vol. 28, no. 2, pp. 183–200, 2002.

[13] S. Wang, K. Pei, J. Whitehouse, J. Yang, and S. Jana, "Formal security analysis of neural networks using symbolic intervals," in *27th USENIX Security Symposium*, 2018, pp. 1599–1614.

[14] S. Gowal, K. Dvijotham, R. Stanforth, R. Bunel, C. Qin, J. Uesato, R. Arandjelovic, T. Mann, and P. Kohli, "On the effectiveness of interval bound propagation for training verifiably robust models," *arXiv preprint arXiv:1810.12715*, 2018.

[15] N. J. Higham, "Accuracy and stability of numerical algorithms (2nd ed.)," *SIAM*, 2002.

[16] B. Paulsen, J. Wang, J. Wang, and C. Wang, "NeuroDiff: Scalable differential verification of neural networks using fine-grained approximation," in *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2020, pp. 784–796.

[17] B. S. Gulavani, T. A. Henzinger, Y. Kannan, A. V. Nori, and S. K. Rajamani, "SYNERGY: A new algorithm for property checking," in *Foundations of Software Engineering (FSE)*, 2006, pp. 117–127.

[18] E. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith, "Counterexample-guided abstraction refinement," in *Computer Aided Verification (CAV)*, 2000, pp. 154–169.

[19] Z. Dong, Z. Yao, A. Gholami, M. W. Mahoney, and K. Keutzer, "HAWQ: Hessian aware quantization of neural networks with mixed-precision," in *Proc. IEEE/CVF ICCV*, 2019.

[20] H. F. Eniser, S. Gerasimou, and A. Sen, "DeepFault: Fault localization for deep neural networks," in *Proc. FASE*, 2019.

[21] S. Ugare, D. Banerjee, S. Misailovic, and G. Singh, "Incremental verification of neural networks," in *Proc. ACM PLDI*, 2023.

[22] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *Proc. ACM ESEC/FSE*, 2021.

[23] H. Zhang, T.-W. Weng, P.-Y. Chen, C.-J. Hsieh, and L. Daniel, "Efficient neural network robustness certification with general activation functions," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[24] M. Giacobbe, T. A. Henzinger, and M. Lechner, "How many bits does it take to quantize your neural network?" in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, 2020, pp. 79–97.

[25] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2011.

[26] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated whitebox testing of deep learning systems," in *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017.

[27] L. Ma, F. Juefei-Xu, F. Zhang, J. Sun, M. Xue, B. Li, C. Chen, T. Su, L. Li, Y. Liu, J. Zhao, and Y. Wang, "DeepGauge: Multi-

granularity testing criteria for deep learning systems," in *Proceedings of the 33rd IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2018.

[28] F. Harel-Canada, L. Wang, M. A. Gulzar, Q. Gu, and M. Kim, "Is neuron coverage a meaningful measure for testing deep neural networks?" in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2020.

[29] P. Godefroid, N. Klarlund, and K. Sen, "DART: Directed automated random testing," in *Programming Language Design and Implementation (PLDI)*, 2005, pp. 213–223.